Some final comments on neural networks. Getting a neural network to converge during training can be tricky. If the network error (ESUM) doesn't steadily decrease, the program must be terminated, changed, and then restarted. This may take several attempts before success is reached. Three things can be changed to affect the convergence: (1) MU, (2) the magnitude of the initial random weights, and (3) the number of hidden nodes (in the order they should be changed).

The most critical item in neural network development is the *validity* of the training examples. For instance, when new commercial products are being developed, the only test data available are from prototypes, simulations, educated guesses, etc. If a neural network is trained on this preliminary information, it might not operate properly in the final application. Any difference between the training database and the eventual data will degrade the neural network's performance (Murphy's law for neural networks). Don't try to second guess the neural network on this issue; you can't!

Recursive Filter Design

Chapters 19 and 20 show how to design recursive filters with the standard frequency responses: high-pass, low-pass, band-pass, etc. What if you need something custom? The answer is to design a recursive filter just as you would a neural network: start with a generic set of recursion coefficients, and use iteration to slowly mold them into what you want. This technique is important for two reasons. First, it allows custom recursive filters to be designed without having to hassle with the mathematics of the z-transform. Second, it shows that the ideas from conventional DSP and neural networks can be combined to form superb algorithms.

The main program for this method is shown in Table 26-4, with two subroutines in Table 26-5. The array, T[ ], holds the desired frequency response, some kind of curve that we have manually designed. Since this program is based around the FFT, the lengths of the signals must be a power of two. As written, this program uses an FFT length of 256, as defined by the variable, N%, in line 130. This means that T[0] to T[128] correspond to the frequencies between 0 and 0.5 of the sampling rate. Only the magnitude is contained in this array; the phase is not controlled in this design, and becomes whatever it becomes.

The recursion coefficients are set to their initial values in lines 270-310, typically selected to be the *identity* system. Don't use random numbers here, or the initial filter will be unstable. The recursion coefficients are held in the arrays, A[ ] and B[ ]. The variable, NP%, sets the number of poles in the designed filter. For example, if NP% is 5, the "a" coefficients run from A[0] to A[5], while the "b" coefficients run from B[1] to B[5].

As previously mentioned, the iterative procedure requires a *single* value that describes how well the current system is functioning. This is provided by the variable, ER (for error), and is calculated in subroutine 3000. Lines